

```
In [4]: from rdflib import Graph
        from collections import defaultdict, Counter

        # Load the RDF data from the file
        rdf_file = "results.ttl"
        g = Graph()
        g.parse(rdf_file, format="turtle")

        # Dictionary to hold property -> Counter of values
        property_value_counts = defaultdict(Counter)

        # Iterate through all triples and count values for each property
        for s, p, o in g:
            property_value_counts[p][o] += 1

        # Print duplicate values for each property
        for prop, values in property_value_counts.items():
            duplicates = {val: count for val, count in values.items() if count > 1}
            print(f"\nProperty: {prop}")
            if duplicates:
                print("  Duplicate values:")
                for val, count in duplicates.items():
                    print(f"    {val} (Count: {count})")
            else:
                print("  No duplicates.")
```

Property: <http://example.org/hasAge>

Duplicate values:

- 48 (Count: 2)
- 69 (Count: 5)
- 56 (Count: 2)
- 34 (Count: 2)
- 75 (Count: 2)
- 68 (Count: 3)
- 29 (Count: 3)
- 76 (Count: 2)
- 61 (Count: 4)
- 63 (Count: 3)
- 65 (Count: 2)
- 74 (Count: 3)
- 42 (Count: 2)
- 71 (Count: 2)
- 22 (Count: 2)

Property: <http://example.org/hasStreet>

No duplicates.

Property: <http://example.org/hasName>

No duplicates.

Property: <http://example.org/hasPostalCode>

No duplicates.

Property: <http://example.org/spokenIn>

Duplicate values:

- <https://dbpedia.org/page/Brazil> (Count: 10)
- <https://dbpedia.org/page/France> (Count: 19)
- <https://dbpedia.org/page/Nigeria> (Count: 12)
- <https://dbpedia.org/page/USA> (Count: 9)
- <https://dbpedia.org/page/Japan> (Count: 10)
- <https://dbpedia.org/page/Italy> (Count: 6)
- <https://dbpedia.org/page/India> (Count: 10)
- <https://dbpedia.org/page/Spain> (Count: 3)
- <https://dbpedia.org/page/UK> (Count: 2)
- <https://dbpedia.org/page/China> (Count: 2)

Property: <http://example.org/takesMedication>

Duplicate values:

- <https://dbpedia.org/page/Salbutamol> (Count: 9)
- <https://dbpedia.org/page/Metformin> (Count: 12)
- <https://dbpedia.org/page/Lisinopril> (Count: 10)
- <https://dbpedia.org/page/Isoniazid> (Count: 14)
- <https://dbpedia.org/page/Artemisinin> (Count: 14)

Property: <http://example.org/hasIncome>

No duplicates.

Property: <http://example.org/hasHairColor>

Duplicate values:

- Gray (Count: 11)
- Black (Count: 14)
- Blonde (Count: 13)

Red (Count: 9)
Brown (Count: 13)

Property: <http://example.org/hasDiseaseDiagnosis>

Duplicate values:

<https://dbpedia.org/page/Asthma> (Count: 12)
<https://dbpedia.org/page/Hypertension> (Count: 19)
<https://dbpedia.org/page/Malaria> (Count: 10)
https://dbpedia.org/page/Diabetes_mellitus (Count: 15)
<https://dbpedia.org/page/Tuberculosis> (Count: 7)

Property: <http://example.org/hasBloodType>

Duplicate values:

<https://www.wikidata.org/wiki/Q19831447> (Count: 13)
<https://www.wikidata.org/wiki/Q19831449> (Count: 17)
<https://www.wikidata.org/wiki/Q19831448> (Count: 15)
<https://www.wikidata.org/wiki/Q19831446> (Count: 15)

Property: <http://example.org/belongsToHospitalSection>

Duplicate values:

<https://www.wikidata.org/wiki/Q56857411> (Count: 29)
<https://www.wikidata.org/wiki/Q56857410> (Count: 21)
<https://www.wikidata.org/wiki/Q56857412> (Count: 8)
<https://www.wikidata.org/wiki/Q56857413> (Count: 2)

Property: <http://example.org/hasCountry>

Duplicate values:

<https://dbpedia.org/page/Nigeria> (Count: 12)
<https://dbpedia.org/page/Japan> (Count: 12)
<https://dbpedia.org/page/USA> (Count: 3)
<https://dbpedia.org/page/India> (Count: 15)
<https://dbpedia.org/page/Brazil> (Count: 9)
<https://dbpedia.org/page/France> (Count: 8)

Property: <http://example.org/hasMaritalStatus>

Duplicate values:

<https://www.wikidata.org/wiki/Q3089372> (Count: 23)
<https://www.wikidata.org/wiki/Q304> (Count: 23)
<https://www.wikidata.org/wiki/Q3089373> (Count: 14)

Property: <http://example.org/hasEducationLevel>

Duplicate values:

<https://www.dbpedia.org/members/master-program> (Count: 19)
<https://www.dbpedia.org/members/bachelor-program> (Count: 20)
<https://www.dbpedia.org/members/doctorate-program> (Count: 21)

Property: <http://example.org/isAllergicTo>

Duplicate values:

https://dbpedia.org/page/Gluten_allergy (Count: 16)
https://dbpedia.org/page/Lactose_intolerance (Count: 14)
https://dbpedia.org/page/Soy_allergy (Count: 19)
https://dbpedia.org/page/Pollen_allergy (Count: 3)
https://dbpedia.org/page/Peanut_allergy (Count: 16)
https://dbpedia.org/page/Dust_mite_allergy (Count: 3)

Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>

Duplicate values:
http://example.org/Person (Count: 60)

Property: http://example.org/hasCity
Duplicate values:
https://dbpedia.org/page/Paris (Count: 12)
https://dbpedia.org/page/Mumbai (Count: 11)
https://dbpedia.org/page/Sao_Paulo (Count: 7)
https://dbpedia.org/page/Tokyo (Count: 17)
https://dbpedia.org/page/Lagos (Count: 12)

Property: http://example.org/hasGeneralPractice
Duplicate values:
Alexander-Palmer (Count: 2)

```
In [5]: from rdflib import Graph
        from collections import defaultdict, Counter
        import matplotlib.pyplot as plt

        # Load RDF graph from file
        rdf_file = "results.ttl"
        g = Graph()
        g.parse(rdf_file, format="turtle")

        # Track value frequencies per property
        property_value_counts = defaultdict(Counter)

        # Count all values for each property
        for s, p, o in g:
            property_value_counts[p][o] += 1

        # Prepare duplicate summary
        duplicate_summary = {}

        for prop, values in property_value_counts.items():
            duplicates = {val: count for val, count in values.items() if count > 1}
            if duplicates:
                duplicate_summary[str(prop)] = duplicates

        # Print summary
        for prop, dup_vals in duplicate_summary.items():
            print(f"\nProperty: {prop}")
            for val, count in dup_vals.items():
                print(f"  {val} (Count: {count})")

        # Visualization: Bar plot per property
        for prop, dup_vals in duplicate_summary.items():
            plt.figure(figsize=(10, 4))
            labels = [str(v) for v in dup_vals.keys()]
            counts = list(dup_vals.values())

            plt.barh(labels, counts, color='skyblue')
            plt.xlabel("Frequency")
            plt.title(f"Duplicate Values for Property: {prop}")
            plt.tight_layout()
            plt.show()
```

Property: <http://example.org/hasAge>

48 (Count: 2)
69 (Count: 5)
56 (Count: 2)
34 (Count: 2)
75 (Count: 2)
68 (Count: 3)
29 (Count: 3)
76 (Count: 2)
61 (Count: 4)
63 (Count: 3)
65 (Count: 2)
74 (Count: 3)
42 (Count: 2)
71 (Count: 2)
22 (Count: 2)

Property: <http://example.org/spokenIn>

<https://dbpedia.org/page/Brazil> (Count: 10)
<https://dbpedia.org/page/France> (Count: 19)
<https://dbpedia.org/page/Nigeria> (Count: 12)
<https://dbpedia.org/page/USA> (Count: 9)
<https://dbpedia.org/page/Japan> (Count: 10)
<https://dbpedia.org/page/Italy> (Count: 6)
<https://dbpedia.org/page/India> (Count: 10)
<https://dbpedia.org/page/Spain> (Count: 3)
<https://dbpedia.org/page/UK> (Count: 2)
<https://dbpedia.org/page/China> (Count: 2)

Property: <http://example.org/takesMedication>

<https://dbpedia.org/page/Salbutamol> (Count: 9)
<https://dbpedia.org/page/Metformin> (Count: 12)
<https://dbpedia.org/page/Lisinopril> (Count: 10)
<https://dbpedia.org/page/Isoniazid> (Count: 14)
<https://dbpedia.org/page/Artemisinin> (Count: 14)

Property: <http://example.org/hasHairColor>

Gray (Count: 11)
Black (Count: 14)
Blonde (Count: 13)
Red (Count: 9)
Brown (Count: 13)

Property: <http://example.org/hasDiseaseDiagnosis>

<https://dbpedia.org/page/Asthma> (Count: 12)
<https://dbpedia.org/page/Hypertension> (Count: 19)
<https://dbpedia.org/page/Malaria> (Count: 10)
https://dbpedia.org/page/Diabetes_mellitus (Count: 15)
<https://dbpedia.org/page/Tuberculosis> (Count: 7)

Property: <http://example.org/hasBloodType>

<https://www.wikidata.org/wiki/Q19831447> (Count: 13)
<https://www.wikidata.org/wiki/Q19831449> (Count: 17)
<https://www.wikidata.org/wiki/Q19831448> (Count: 15)
<https://www.wikidata.org/wiki/Q19831446> (Count: 15)

Property: <http://example.org/belongsToHospitalSection>
<https://www.wikidata.org/wiki/Q56857411> (Count: 29)
<https://www.wikidata.org/wiki/Q56857410> (Count: 21)
<https://www.wikidata.org/wiki/Q56857412> (Count: 8)
<https://www.wikidata.org/wiki/Q56857413> (Count: 2)

Property: <http://example.org/hasCountry>
<https://dbpedia.org/page/Nigeria> (Count: 12)
<https://dbpedia.org/page/Japan> (Count: 12)
<https://dbpedia.org/page/USA> (Count: 3)
<https://dbpedia.org/page/India> (Count: 15)
<https://dbpedia.org/page/Brazil> (Count: 9)
<https://dbpedia.org/page/France> (Count: 8)

Property: <http://example.org/hasMaritalStatus>
<https://www.wikidata.org/wiki/Q3089372> (Count: 23)
<https://www.wikidata.org/wiki/Q304> (Count: 23)
<https://www.wikidata.org/wiki/Q3089373> (Count: 14)

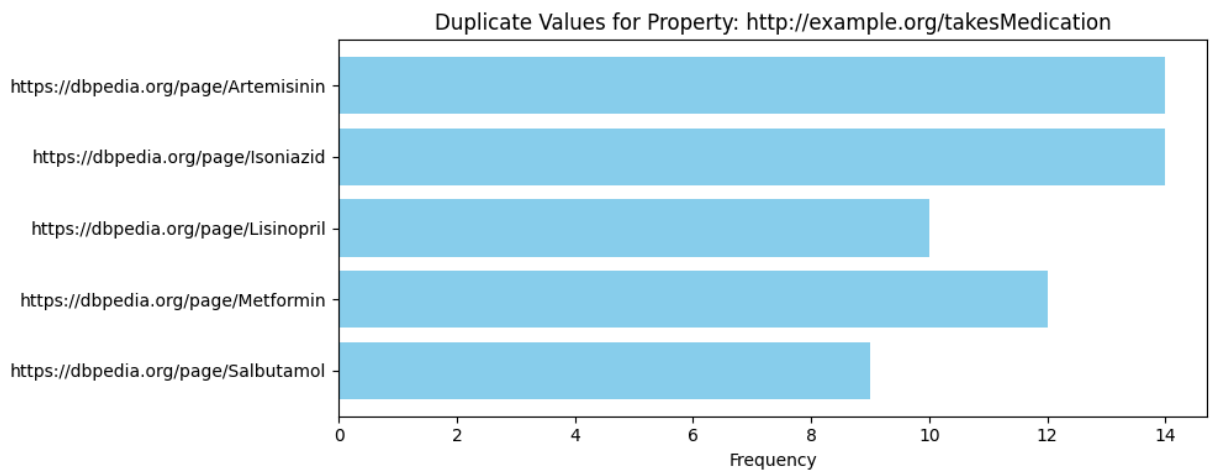
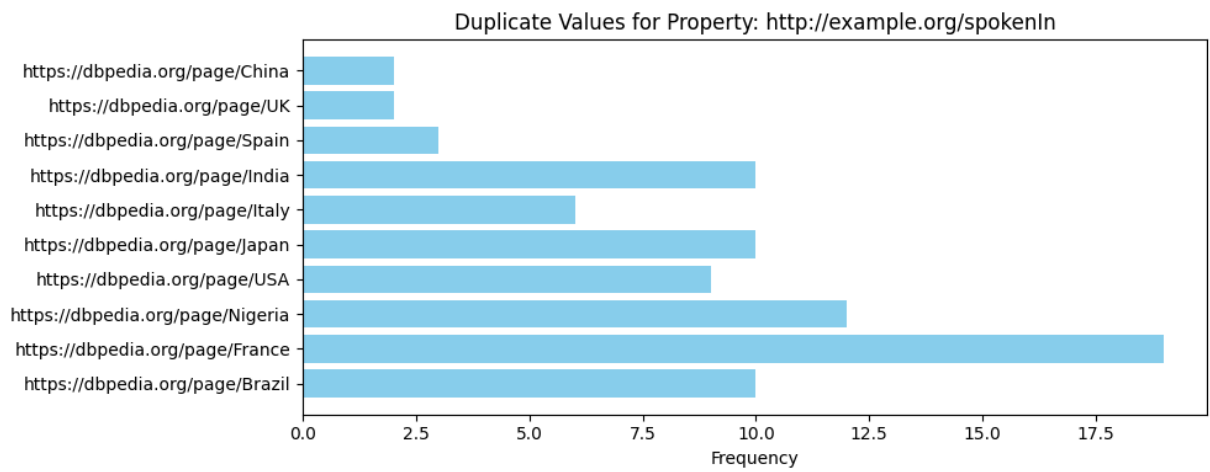
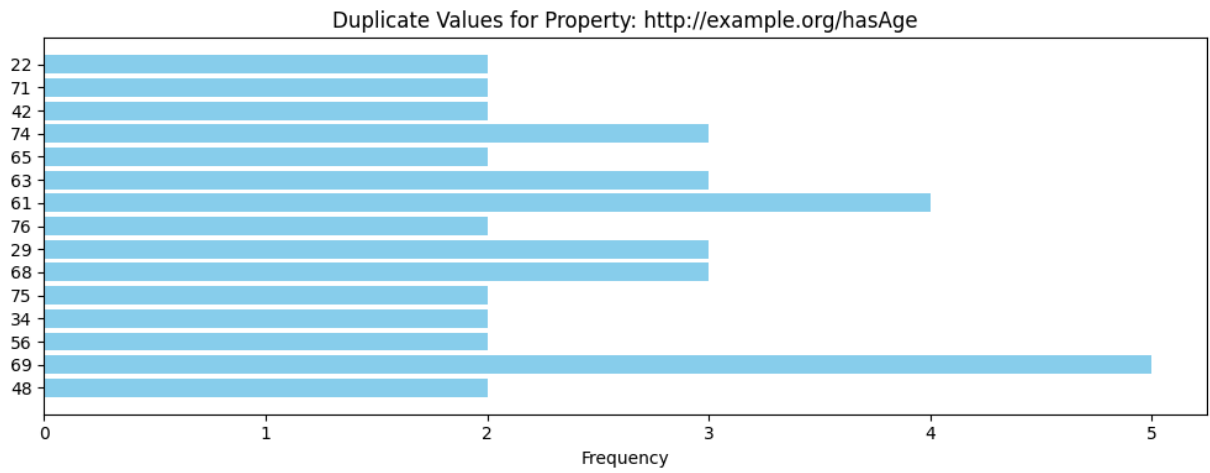
Property: <http://example.org/hasEducationLevel>
<https://www.dbpedia.org/members/master-program> (Count: 19)
<https://www.dbpedia.org/members/bachelor-program> (Count: 20)
<https://www.dbpedia.org/members/doctorate-program> (Count: 21)

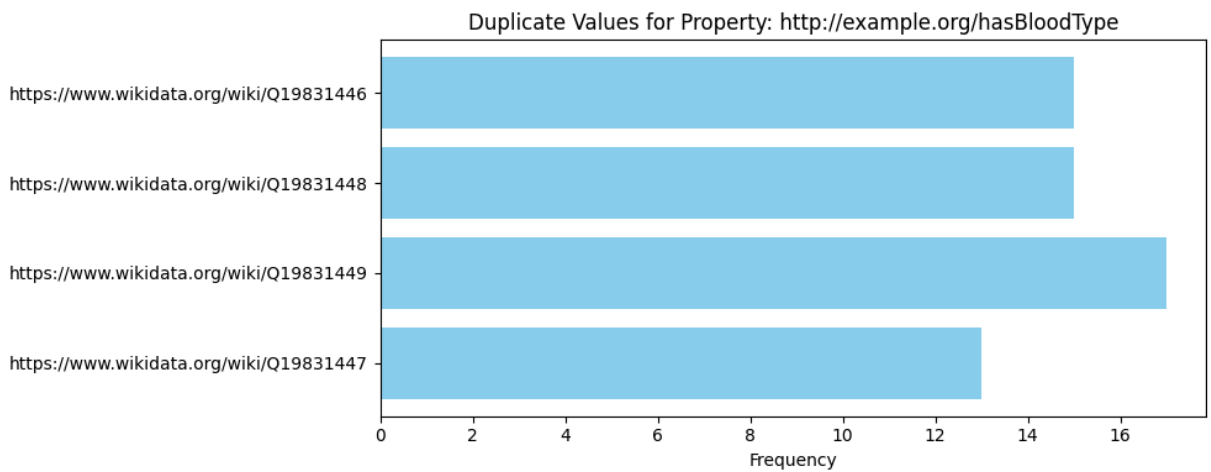
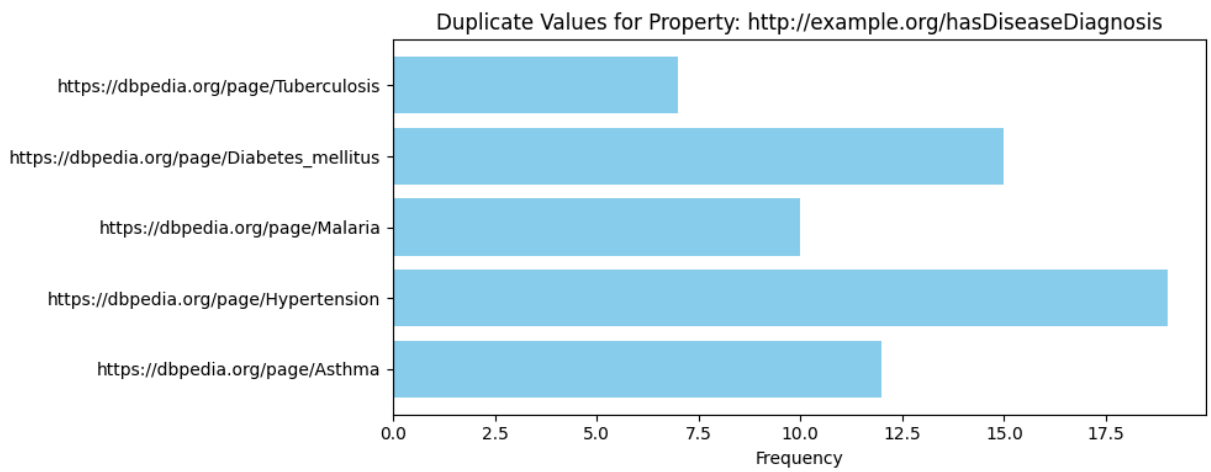
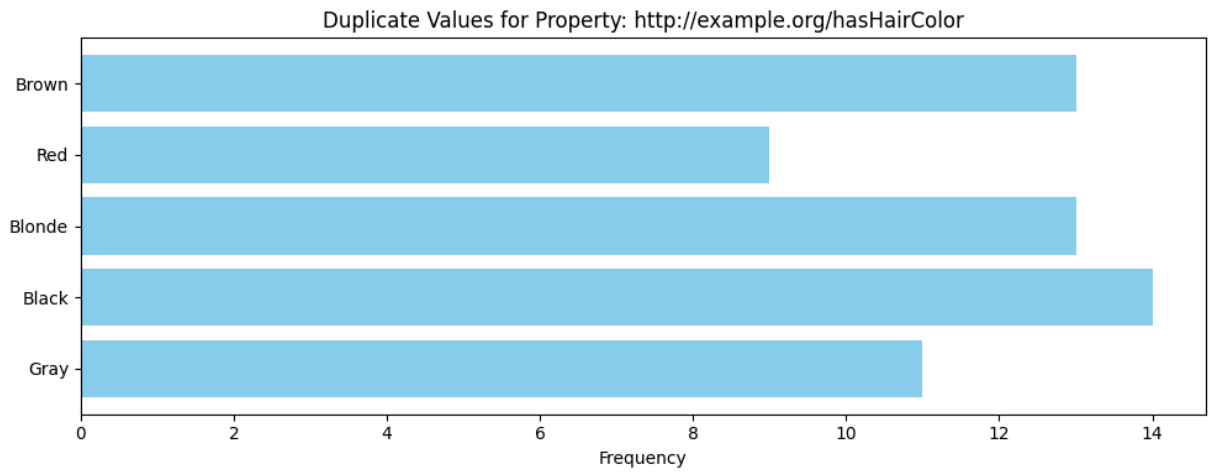
Property: <http://example.org/isAllergicTo>
https://dbpedia.org/page/Gluten_allergy (Count: 16)
https://dbpedia.org/page/Lactose_intolerance (Count: 14)
https://dbpedia.org/page/Soy_allergy (Count: 19)
https://dbpedia.org/page/Pollen_allergy (Count: 3)
https://dbpedia.org/page/Peanut_allergy (Count: 16)
https://dbpedia.org/page/Dust_mite_allergy (Count: 3)

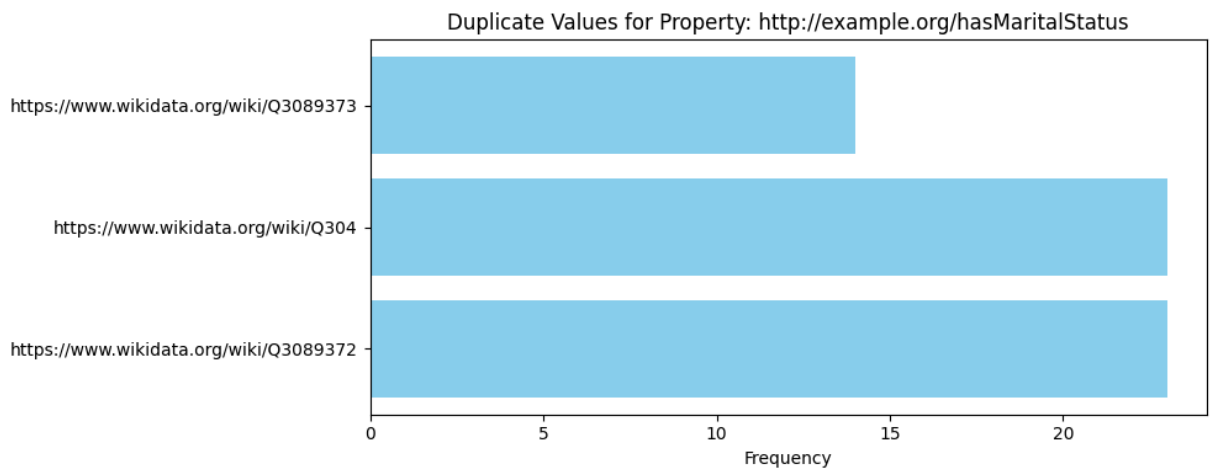
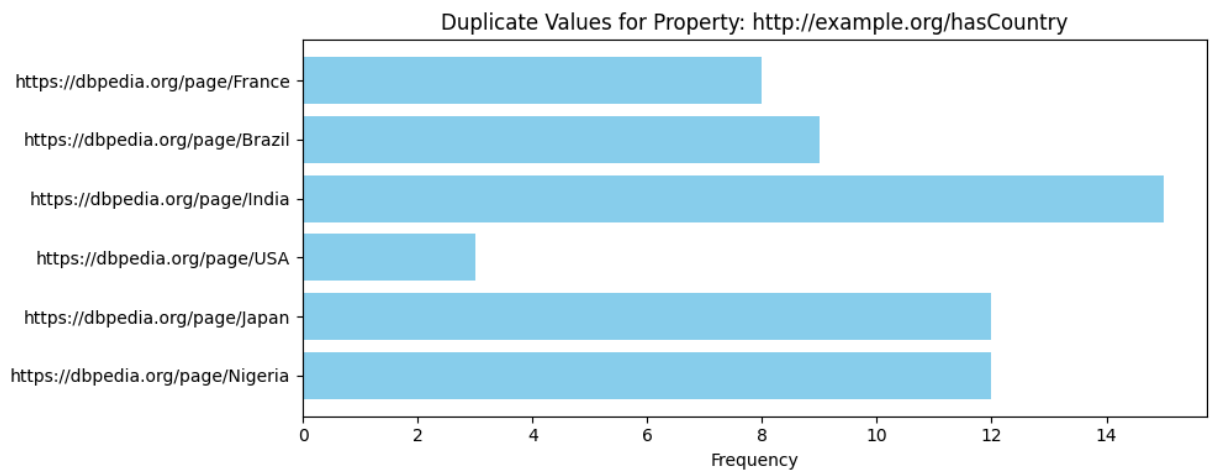
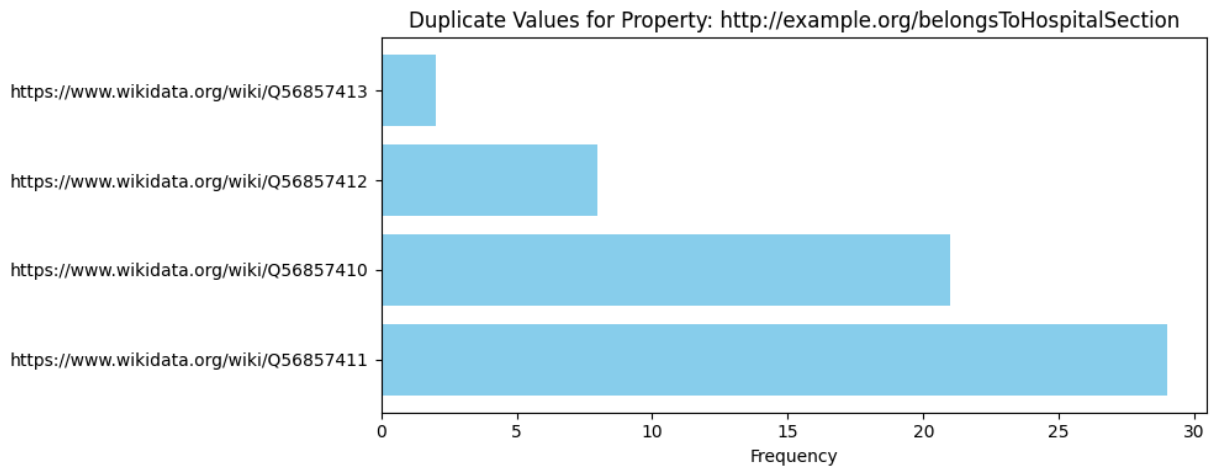
Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<http://example.org/Person> (Count: 60)

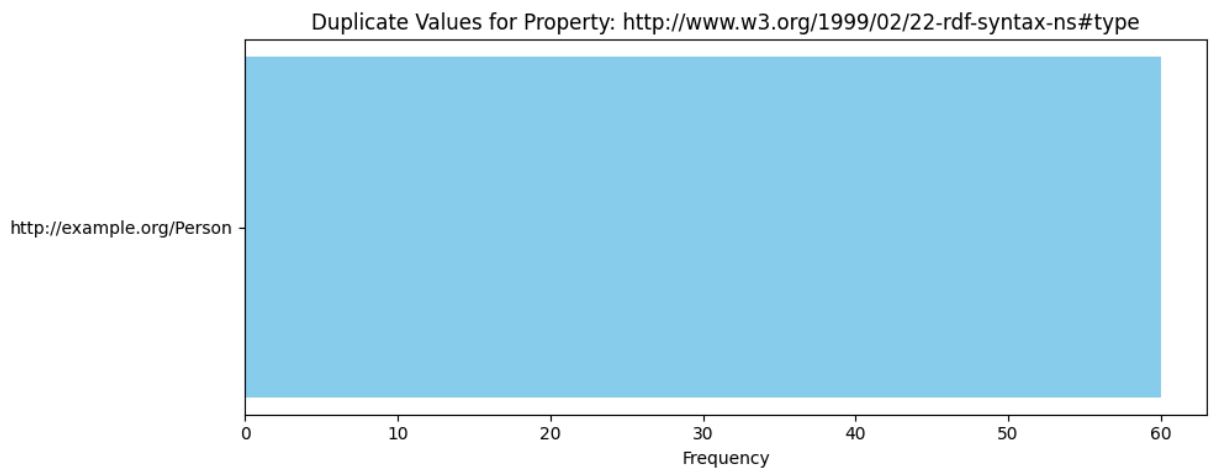
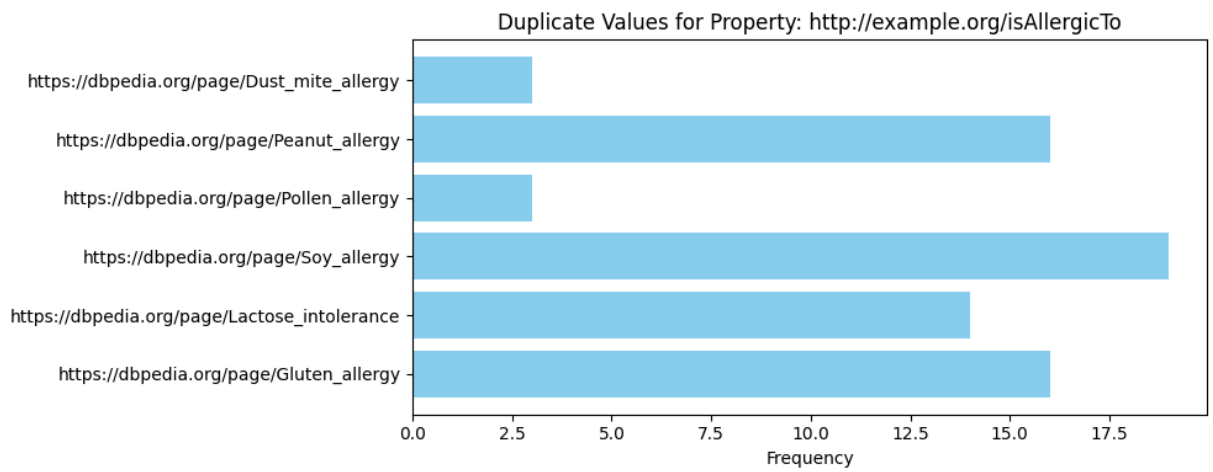
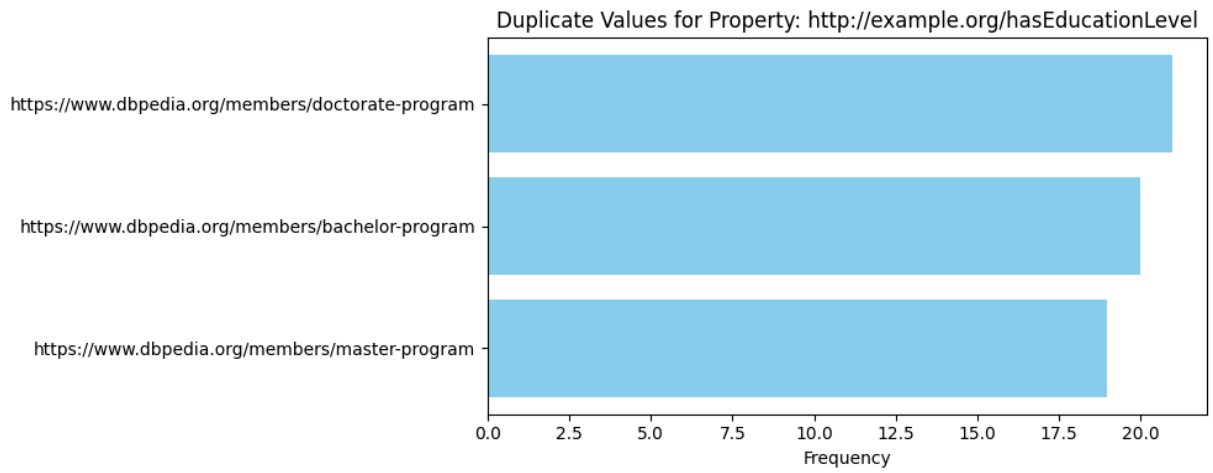
Property: <http://example.org/hasCity>
<https://dbpedia.org/page/Paris> (Count: 12)
<https://dbpedia.org/page/Mumbai> (Count: 11)
https://dbpedia.org/page/Sao_Paulo (Count: 7)
<https://dbpedia.org/page/Tokyo> (Count: 17)
<https://dbpedia.org/page/Lagos> (Count: 12)

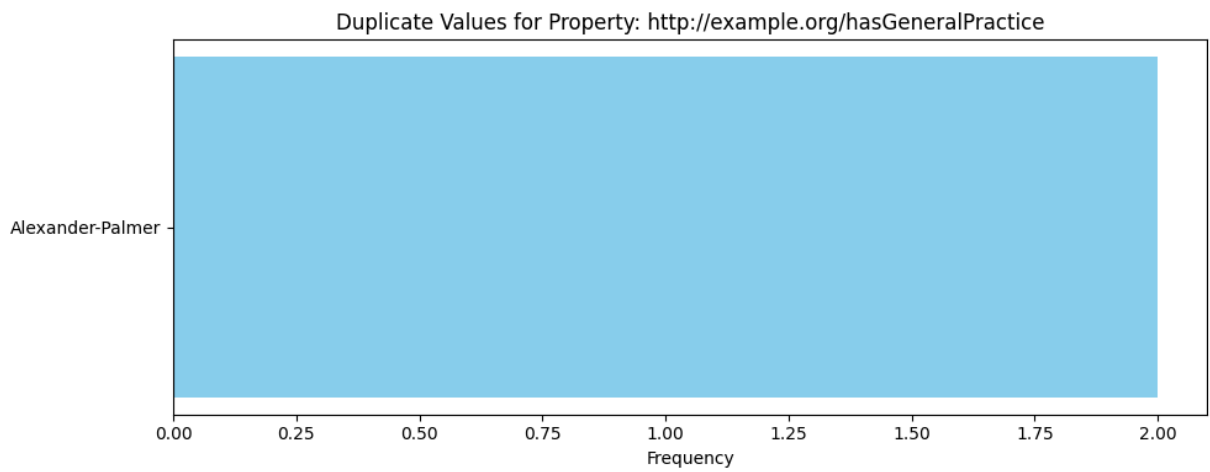
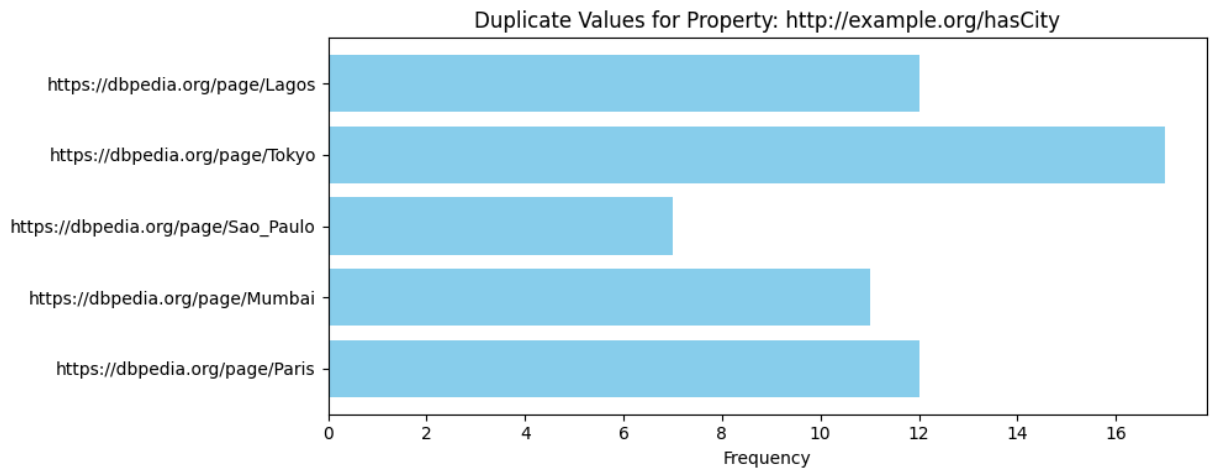
Property: <http://example.org/hasGeneralPractice>
[Alexander-Palmer](#) (Count: 2)











```
In [6]: from rdflib import Graph
import pandas as pd
import numpy as np
from collections import defaultdict

g = Graph()
g.parse("results.ttl", format="turtle") # Replace with your file or string

# Collect data
data = defaultdict(dict)

for s, p, o in g:
    subj = str(s)
    pred = str(p).split("/")[-1] # Simplify URI to predicate name
    obj = str(o)

    # Handle multiple values (like isAllergicTo, spokenIn)
    if pred in data[subj]:
        if isinstance(data[subj][pred], list):
            data[subj][pred].append(obj)
        else:
            data[subj][pred] = [data[subj][pred], obj]
    else:
        data[subj][pred] = obj

df = pd.DataFrame.from_dict(data, orient='index')
```

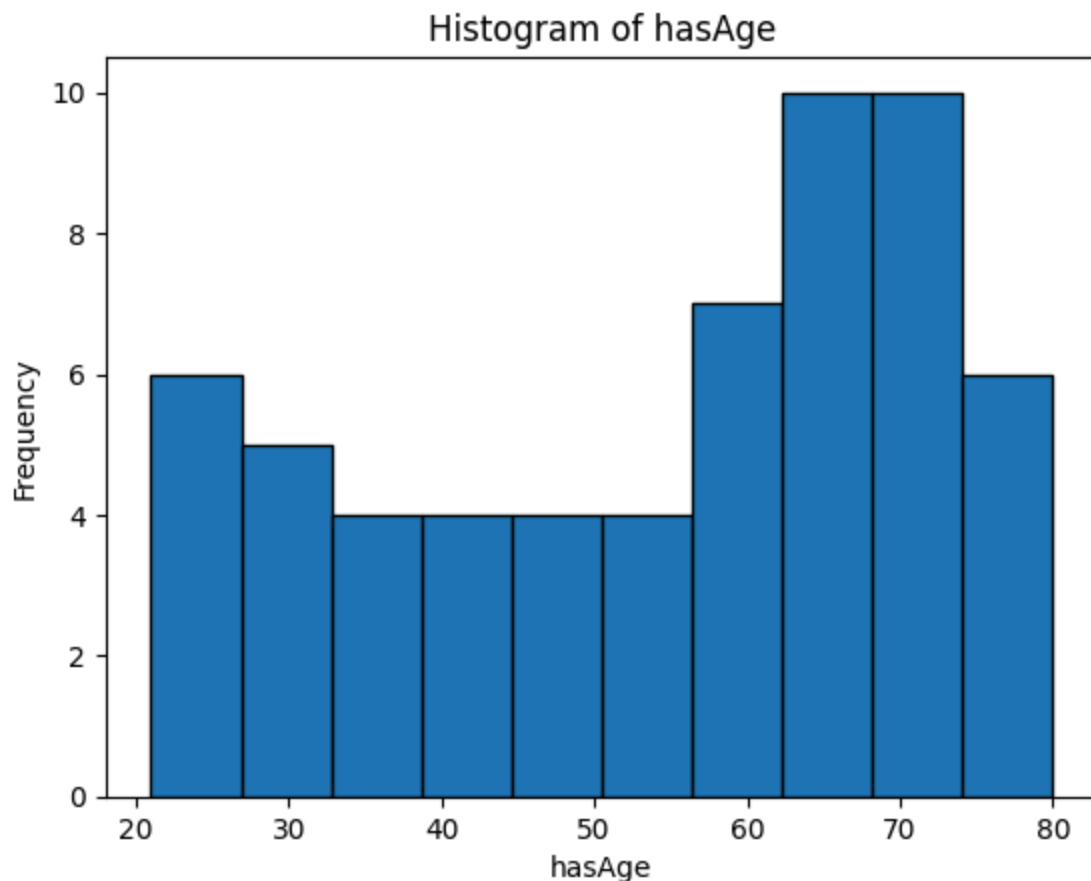
```
In [7]: import matplotlib.pyplot as plt
from scipy.stats import skew, normaltest

def analyze_numeric(col):
    values = pd.to_numeric(df[col], errors='coerce').dropna()
    print(f"Column: {col}")
    print(f"Skewness: {skew(values)}")
    k2, p = normaltest(values)
    print(f"Normality test p-value: {p}")
    print("Distribution: ", "Likely normal" if p > 0.05 else "Not normal (sk

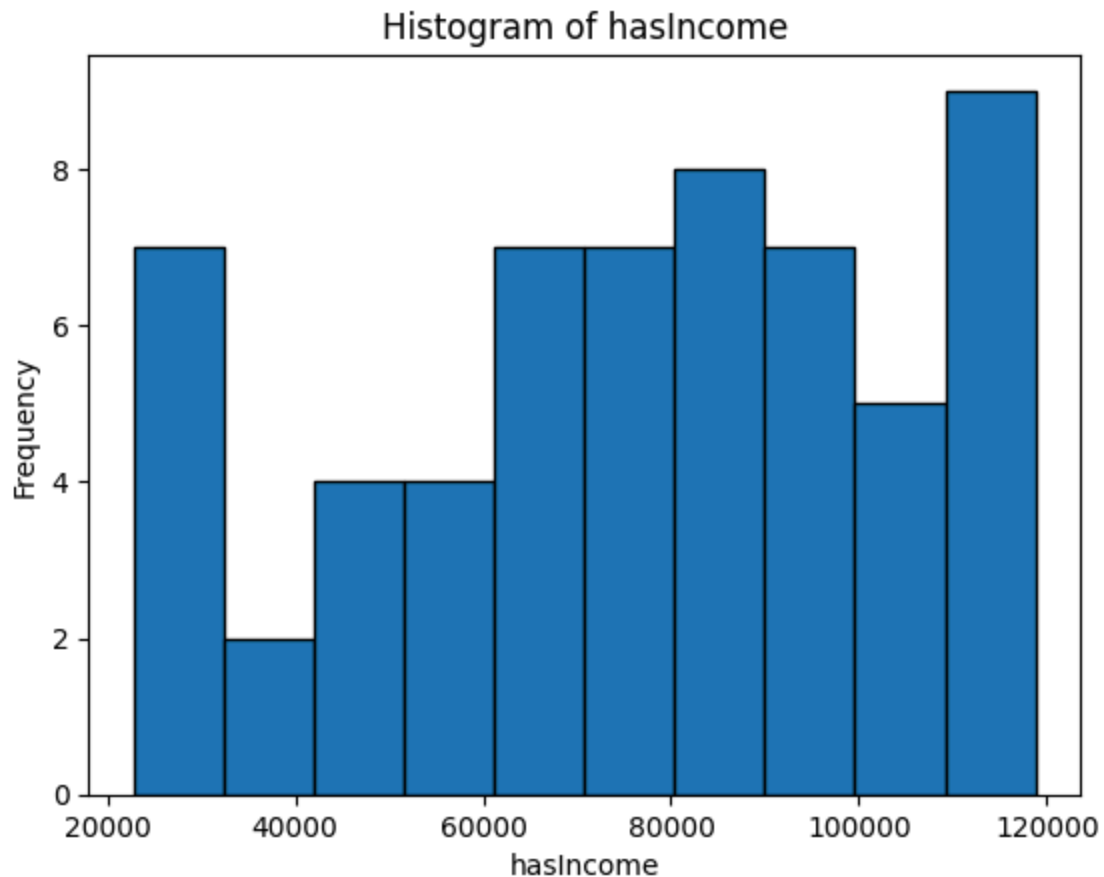
    plt.hist(values, bins=10, edgecolor='black')
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()

analyze_numeric('hasAge')
analyze_numeric('hasIncome')
```

Column: hasAge
 Skewness: -0.4742478071174032
 Normality test p-value: 0.0007753544736736232
 Distribution: Not normal (skewed)



Column: hasIncome
 Skewness: -0.34939076624857696
 Normality test p-value: 0.08463696079678804
 Distribution: Likely normal



```
In [8]: def analyze_categorical(col):
        series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list))
        counts = series.value_counts()
        print(f"Column: {col}")
        print(counts)
        counts.plot(kind='bar', title=f"Distribution of {col}")
        plt.ylabel("Count")
        plt.show()

        categorical_cols = ['hasBloodType', 'hasCountry', 'hasHairColor', 'spokenIn']
        for col in categorical_cols:
            analyze_categorical(col)
```

Column: hasBloodType

hasBloodType

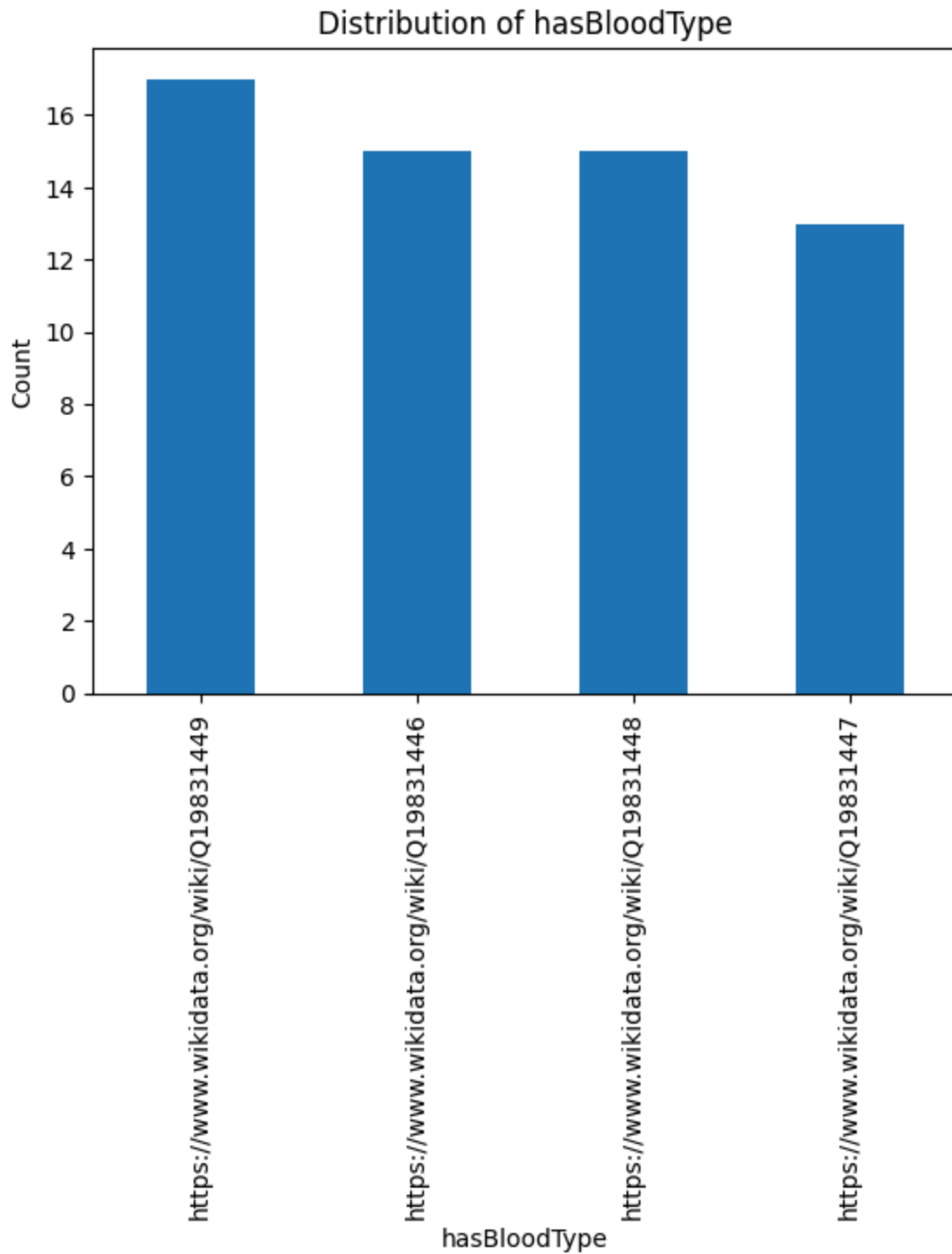
<https://www.wikidata.org/wiki/Q19831449> 17

<https://www.wikidata.org/wiki/Q19831446> 15

<https://www.wikidata.org/wiki/Q19831448> 15

<https://www.wikidata.org/wiki/Q19831447> 13

Name: count, dtype: int64

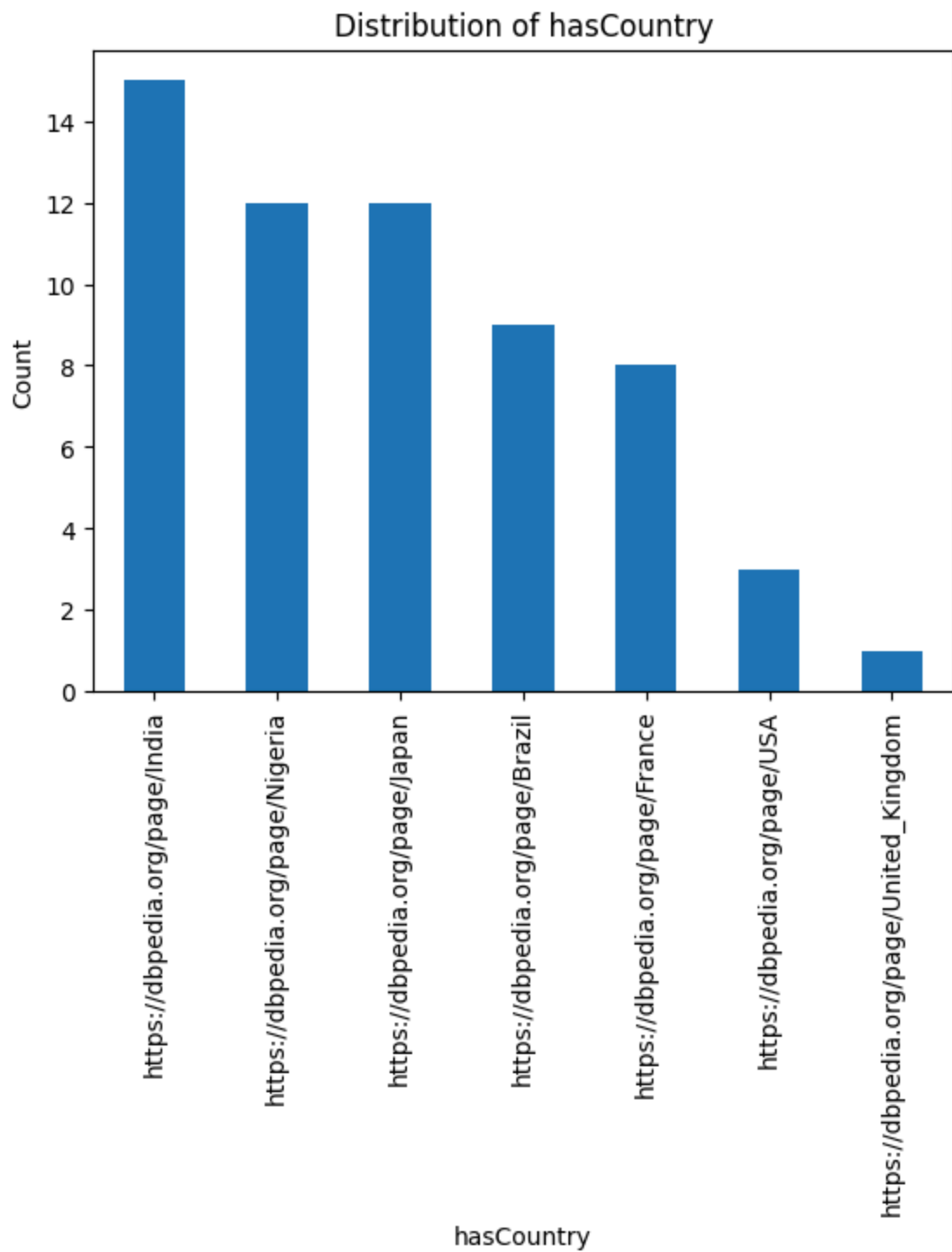


Column: hasCountry

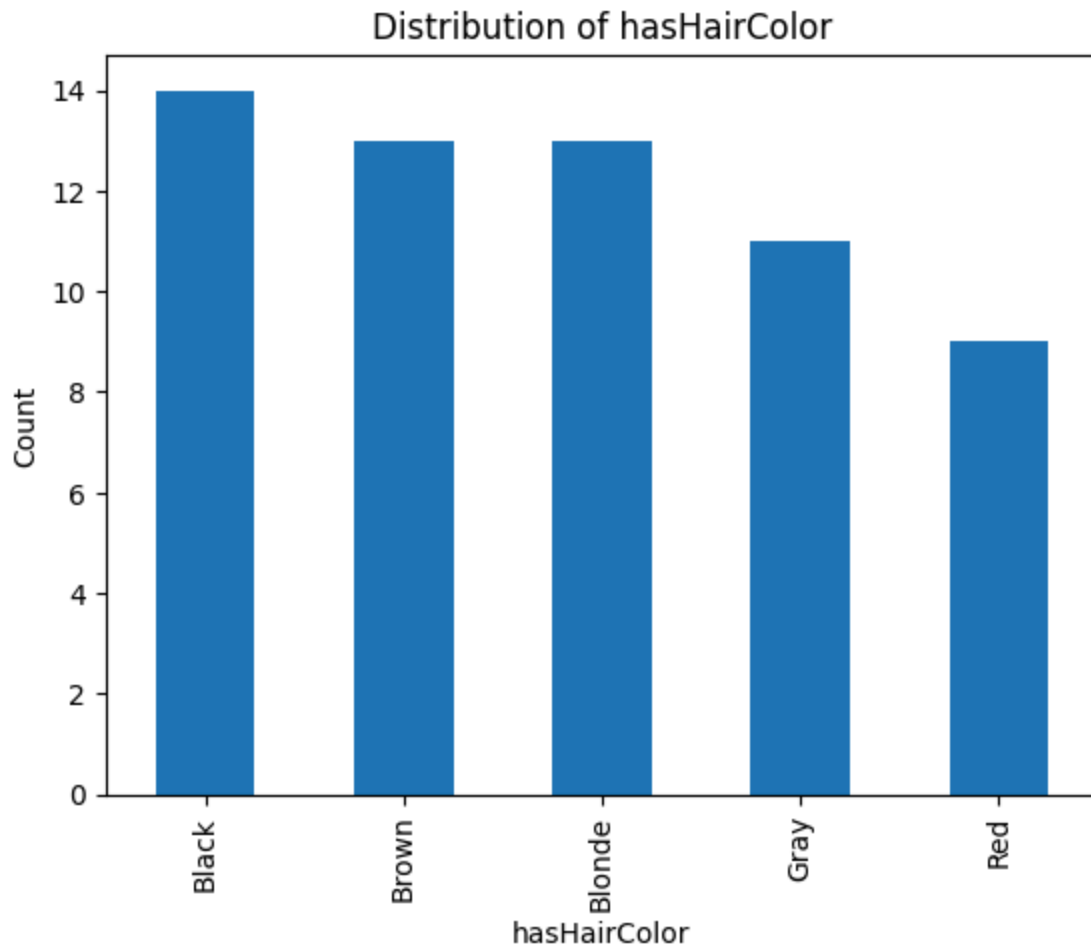
hasCountry

https://dbpedia.org/page/India	15
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	12
https://dbpedia.org/page/Brazil	9
https://dbpedia.org/page/France	8
https://dbpedia.org/page/USA	3
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64



Column: hasHairColor
hasHairColor
Black 14
Brown 13
Blonde 13
Gray 11
Red 9
Name: count, dtype: int64

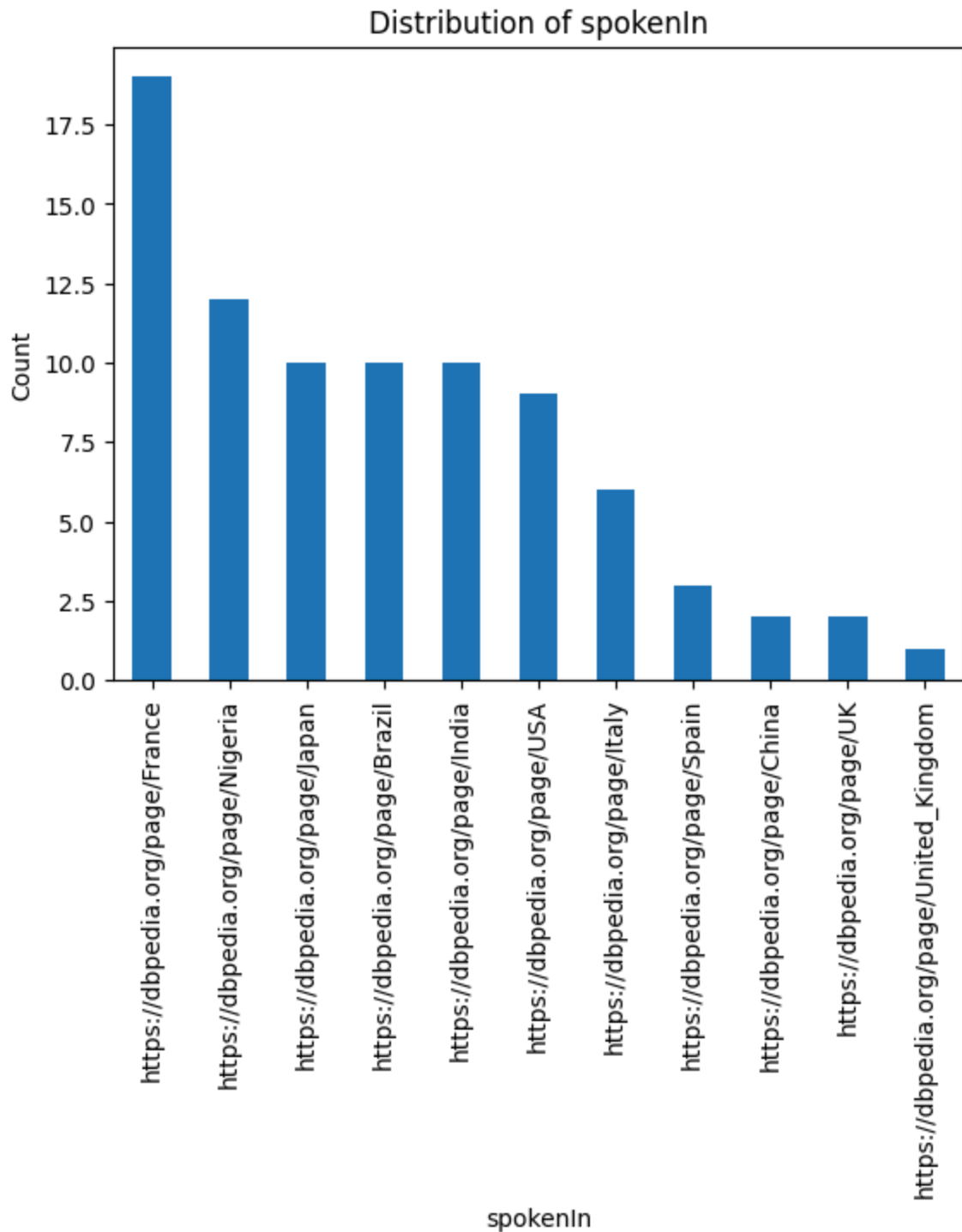


Column: spokenIn

spokenIn

https://dbpedia.org/page/France	19
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	10
https://dbpedia.org/page/Brazil	10
https://dbpedia.org/page/India	10
https://dbpedia.org/page/USA	9
https://dbpedia.org/page/Italy	6
https://dbpedia.org/page/Spain	3
https://dbpedia.org/page/China	2
https://dbpedia.org/page/UK	2
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

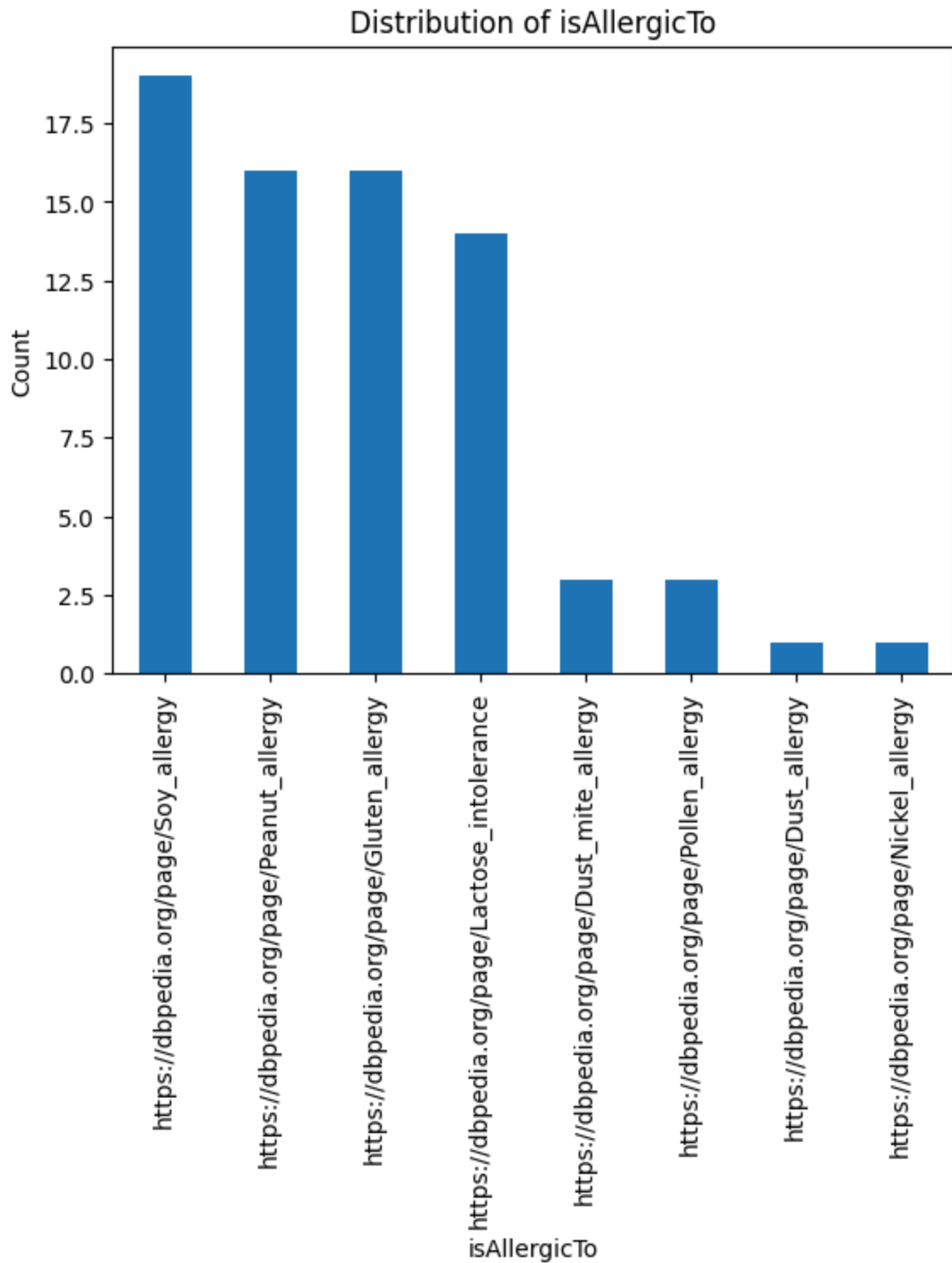


Column: isAllergicTo

isAllergicTo

https://dbpedia.org/page/Soy_allergy	19
https://dbpedia.org/page/Peanut_allergy	16
https://dbpedia.org/page/Gluten_allergy	16
https://dbpedia.org/page/Lactose_intolerance	14
https://dbpedia.org/page/Dust_mite_allergy	3
https://dbpedia.org/page/Pollen_allergy	3
https://dbpedia.org/page/Dust_allergy	1
https://dbpedia.org/page/Nickel_allergy	1

Name: count, dtype: int64



```
In [19]: import rdflib
import requests
import openai
from urllib.parse import urlparse
from collections import defaultdict
import time

# Hardcoded Groq/OpenAI-compatible API configuration
openai.api_key = "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"
openai.api_base = "https://api.groq.com/openai/v1"
```

```

# Property descriptions for relevance checks
property_prompts = {
    "hasBloodType": "is a valid bloodtype?",
    "hasAge": "is a valid age?",
    "belongsToHospitalSection": "is a valid hospital section?",
    "hasCity": "is a real city?",
    "hasCountry": "is a real country?",
    "hasDiseaseDiagnosis": "is a disease?",
    "hasEducationLevel": "is a degree?",
    "hasGeneralPractice": "is a person's name?",
    "hasHairColor": "is a color?",
    "hasIncome": "is an integer?",
    "hasMaritalStatus": "is marital status?",
    "hasName": "is a person's name?",
    "hasPostalCode": "is a valid postal code?",
    "hasStreet": "is a real street?",
    "isAllergicTo": "is a type of allergy?",
    "spokenIn": "is a country?"
}

# Load the RDF data
g = rdflib.Graph()
g.parse("results.ttl", format="ttl")

# Validate if a URI exists (returns True/False)
def is_uri_valid(uri):
    try:
        response = requests.head(uri, timeout=5)
        return response.status_code == 200
    except requests.exceptions.RequestException:
        return False

# Extract URIs grouped by property (predicate)
def extract_uris_by_property(graph):
    uris_by_predicate = defaultdict(set)
    for subj, pred, obj in graph:
        if isinstance(obj, rdflib.URIRef):
            pred_name = pred.split("#")[-1] if "#" in pred else pred.split("/")[-1]
            uris_by_predicate[pred_name].add(str(obj))
    return uris_by_predicate

# Relevance check using Groq LLM (OpenAI-compatible)
def check_relevance_with_ai(uri, prompt_text):
    parsed = urlparse(uri)
    entity = parsed.path.strip("/").split("/")[-1].replace("_", " ")
    prompt = f"'{entity}' {prompt_text} Answer only with True or False."

    try:
        response = openai.ChatCompletion.create(
            model="meta-llama/llama-4-scout-17b-16e-instruct",
            messages=[
                {"role": "system", "content": "Answer only with True or False"},
                {"role": "user", "content": prompt}
            ],
            temperature=0
        )
    
```

```

    )
    reply = response['choices'][0]['message']['content'].strip().lower()
    return reply.startswith("true")
except Exception as e:
    print(f"Error checking relevance for {uri}: {e}")
    return False

# Run validity and relevance checks
def validate_and_check_all(graph):
    uris_by_property = extract_uris_by_property(graph)
    total_counts = {}

    for prop, uris in uris_by_property.items():
        print(f"\nChecking property: {prop}")
        prompt_text = property_prompts.get(prop, "is this a relevant entity?")
        valid_and_relevant_count = 0

        for uri in uris:
            valid = is_uri_valid(uri)
            relevant = check_relevance_with_ai(uri, prompt_text)
            print(f"URI: {uri}\n → Valid: {valid}, Relevant: {relevant}")

            if valid and relevant:
                valid_and_relevant_count += 1

        time.sleep(5) # Delay of 5 seconds after each request

    total_counts[prop] = valid_and_relevant_count
    print(f"✅ Count of Valid & Relevant URIs for {prop}: {valid_and_relevant_count}")

print("\n📊 Final Summary:")
for prop, count in total_counts.items():
    print(f"{prop}: {count} valid & relevant URIs")

# Execute
validate_and_check_all(g)

```

Checking property: spokenIn

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/UK>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Italy>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/China>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Spain>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for spokenIn: 9

Checking property: takesMedication

URI: <https://dbpedia.org/page/Metformin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Isoniazid>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Artemisinin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Insulin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Salbutamol>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Lisinopril>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for takesMedication: 6

Checking property: hasDiseaseDiagnosis

URI: https://dbpedia.org/page/Diabetes_mellitus

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Hypertension>

→ Valid: True, Relevant: False

URI: <https://dbpedia.org/page/Malaria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Tuberculosis>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Asthma>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for hasDiseaseDiagnosis: 3

Checking property: hasBloodType

URI: <https://www.wikidata.org/wiki/Q19831446>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831448>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q19831447>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q19831449>
→ Valid: True, Relevant: False
✅ Count of Valid & Relevant URIs for hasBloodType: 0

Checking property: belongsToHospitalSection
URI: <https://www.wikidata.org/wiki/Q56857412>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857410>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857411>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857413>
→ Valid: True, Relevant: False
✅ Count of Valid & Relevant URIs for belongsToHospitalSection: 0

Checking property: hasCountry
URI: <https://dbpedia.org/page/Nigeria>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/India>
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/United_Kingdom
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/Brazil>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/France>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/USA>
→ Valid: False, Relevant: True
URI: <https://dbpedia.org/page/Japan>
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for hasCountry: 6

Checking property: hasMaritalStatus
URI: <https://www.wikidata.org/wiki/Q3089373>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q3089372>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q304>
→ Valid: False, Relevant: False
✅ Count of Valid & Relevant URIs for hasMaritalStatus: 0

Checking property: hasEducationLevel
URI: <https://www.dbpedia.org/members/bachelor-program>
→ Valid: False, Relevant: False
URI: <https://www.dbpedia.org/members/master-program>
→ Valid: False, Relevant: False
URI: <https://www.dbpedia.org/members/doctorate-program>
→ Valid: False, Relevant: False
✅ Count of Valid & Relevant URIs for hasEducationLevel: 0

Checking property: isAllergicTo
URI: https://dbpedia.org/page/Gluten_allergy

```

    → Valid: False, Relevant: False
URI: https://dbpedia.org/page/Soy_allergy
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Dust_mite_allergy
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Lactose_intolerance
    → Valid: True, Relevant: False
URI: https://dbpedia.org/page/Peanut_allergy
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Pollen_allergy
    → Valid: False, Relevant: True
URI: https://dbpedia.org/page/Dust_allergy
    → Valid: False, Relevant: True
URI: https://dbpedia.org/page/Nickel_allergy
    → Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for isAllergicTo: 4

```

```

Checking property: type
URI: http://example.org/Person
    → Valid: False, Relevant: True
✅ Count of Valid & Relevant URIs for type: 0

```

```

Checking property: hasCity
URI: https://dbpedia.org/page/Mumbai
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Tokyo
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Lagos
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/New_York
    → Valid: True, Relevant: True
URI: https://dbpedia.org/page/Sao_Paulo
    → Valid: False, Relevant: True
URI: https://dbpedia.org/page/Paris
    → Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for hasCity: 5

```

```

📊 Final Summary:
spokenIn: 9 valid & relevant URIs
takesMedication: 6 valid & relevant URIs
hasDiseaseDiagnosis: 3 valid & relevant URIs
hasBloodType: 0 valid & relevant URIs
belongsToHospitalSection: 0 valid & relevant URIs
hasCountry: 6 valid & relevant URIs
hasMaritalStatus: 0 valid & relevant URIs
hasEducationLevel: 0 valid & relevant URIs
isAllergicTo: 4 valid & relevant URIs
type: 0 valid & relevant URIs
hasCity: 5 valid & relevant URIs

```

```

In [ ]: import rdflib
import requests
import openai
from urllib.parse import urlparse
from collections import defaultdict
import time

```

```

# Hardcoded Groq/OpenAI-compatible API configuration
openai.api_key = "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"
openai.api_base = "https://api.groq.com/openai/v1"

# Property descriptions for relevance checks
property_prompts = {
    "hasBloodType": "is a valid bloodtype?",
    "hasAge": "is a valid age?",
    "belongsToHospitalSection": "is a valid hospital section?",
    "hasCity": "is a real city?",
    "hasCountry": "is a real country?",
    "hasDiseaseDiagnosis": "is a disease?",
    "hasEducationLevel": "is a reference to education degree?",
    "hasGeneralPractice": "is a person's name?",
    "hasHairColor": "is a color?",
    "hasIncome": "is an integer?",
    "hasMaritalStatus": "is a reference to marital status?",
    "hasName": "is a person's name?",
    "hasPostalCode": "is a valid postal code?",
    "hasStreet": "is a real street?",
    "isAllergicTo": "is a type of allergy?",
    "spokenIn": "is a country?"
}

# Load the RDF data
g = rdflib.Graph()
g.parse("results.ttl", format="ttl")

# Validate if a URI exists (returns True/False)
def is_uri_valid(uri):
    try:
        response = requests.head(uri, timeout=5)
        return response.status_code == 200
    except requests.exceptions.RequestException:
        return False

# Extract URIs grouped by property (predicate)
def extract_uris_by_property(graph):
    uris_by_predicate = defaultdict(set)
    for subj, pred, obj in graph:
        if isinstance(obj, rdflib.URIRef):
            pred_name = pred.split("#")[-1] if "#" in pred else pred.split("/")[-1]
            uris_by_predicate[pred_name].add(str(obj))
    return uris_by_predicate

# Relevance check using Groq LLM (OpenAI-compatible)
def check_relevance_with_ai(uri, prompt_text):
    parsed = urlparse(uri)
    entity = parsed.path.strip("/").split("/")[-1].replace("_", " ")
    prompt = f"'{entity}' {prompt_text} Answer only with True or False."

    try:
        response = openai.ChatCompletion.create(
            model="meta-llama/llama-4-scout-17b-16e-instruct",
            messages=[

```

```

        {"role": "system", "content": "Answer only with True or False"}
        {"role": "user", "content": prompt}
    ],
    temperature=0
)
reply = response['choices'][0]['message']['content'].strip().lower()
return reply.startswith("true")
except Exception as e:
    print(f"Error checking relevance for {uri}: {e}")
    return False

# Run validity and relevance checks
def validate_and_check_all(graph):
    uris_by_property = extract_uris_by_property(graph)
    total_counts = {}

    for prop, uris in uris_by_property.items():
        print(f"\nChecking property: {prop}")
        prompt_text = property_prompts.get(prop, "is this a relevant entity?")
        valid_and_relevant_count = 0

        for uri in uris:
            valid = is_uri_valid(uri)
            relevant = check_relevance_with_ai(uri, prompt_text)
            print(f"URI: {uri}\n → Valid: {valid}, Relevant: {relevant}")

            if valid and relevant:
                valid_and_relevant_count += 1

        time.sleep(5) # Delay of 5 seconds after each request

    total_counts[prop] = valid_and_relevant_count
    print(f"✅ Count of Valid & Relevant URIs for {prop}: {valid_and_relevant_count}")

print("\n📊 Final Summary:")
for prop, count in total_counts.items():
    print(f"{prop}: {count} valid & relevant URIs")

# Execute
validate_and_check_all(g)

```

Checking property: hasEducationLevel

URI: <https://www.wikidata.org/wiki/Q163727>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q183816>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q849697>

→ Valid: True, Relevant: False

✅ Count of Valid & Relevant URIs for hasEducationLevel: 0

Checking property: hasMaritalStatus

URI: <https://www.wikidata.org/wiki/Q28341938>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q179115>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q37713312>

→ Valid: True, Relevant: False

✅ Count of Valid & Relevant URIs for hasMaritalStatus: 0

Checking property: spokenIn

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/UK>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Italy>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/China>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Spain>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for spokenIn: 9

Checking property: takesMedication

URI: <https://dbpedia.org/page/Metformin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Isoniazid>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Artemisinin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Insulin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Salbutamol>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Lisinopril>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for takesMedication: 6

Checking property: hasBloodType

URI: <https://www.wikidata.org/wiki/Q19831454>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831453>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831449>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831455>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for hasBloodType: 0

Checking property: hasDiseaseDiagnosis

URI: https://dbpedia.org/page/Diabetes_mellitus

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Hypertension>

→ Valid: True, Relevant: False

URI: <https://dbpedia.org/page/Malaria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Tuberculosis>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Asthma>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for hasDiseaseDiagnosis: 3

Checking property: belongsToHospitalSection

URI: <https://www.wikidata.org/wiki/Q56857412>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857410>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857411>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857413>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for belongsToHospitalSection: 0

Checking property: hasCountry

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for hasCountry: 6

Checking property: isAllergicTo

URI: https://dbpedia.org/page/Gluten_allergy

→ Valid: False, Relevant: False
URI: https://dbpedia.org/page/Soy_allergy
→ Valid: True, Relevant: True

In []:

In []: